

FNAF_RTS — Documento técnico del proyecto

1. Visión general

FNAF_RTS es un juego 2D en Java con Swing que mezcla la tensión de Five Nights at Freddy's con decisiones tácticas tipo RTS. El jugador administra puertas, luces y cámaras desde una oficina mientras 6 animatrónicos se mueven por un grafo de nodos con IA propia. El objetivo es sobrevivir de las 12 AM a las 6 AM.

Stack: Java 17, Swing, Maven. Sin dependencias externas.

Arquitectura: MVC ligero con separación estricta modelo / vista / control.

Concurrencia: ScheduledExecutorService para el loop, synchronized con lock único para el estado.

2. Estructura de paquetes

text

```
• fnafrts/
• |— Main.java           Punto de entrada, bucle menú-juego
• |— core/
• |   |— GameState.java   Estado global, tick, log
• |   |— GameLoop.java    Scheduler a 10 Hz con timeScale
• |   |— AnimatronicView.java Snapshot inmutable para la vista
• |— model/
• |   |— animatronic/
• |   |   |— Animatronic.java Clase base + hooks
• |   |   |— Chica.java       Pizza y cocina
• |   |   |— Bonnie.java     Desvío a H16 y bloqueo de cámaras
• |   |   |— Freddy.java     Congelado al observar, blink
• |   |   |— Foxy.java       Rush desde la entrada
• |   |   |— GoldenFreddy.java Teletransporte y apagón
• |   |   |— Endo.java       Shock y modo silencioso
• |   |— graph/
• |   |   |— MapGraph.java    Grafo con BFS cacheado
• |   |   |— Node.java        Nodo con tipo y vecinos
• |   |   |— NodeType.java    NORMAL, OFFICE, ENTRY_LEFT, ENTRY_RIGHT
• |   |   |— Sector.java     Agrupación lógica de nodos
• |— systems/
```

- | | — AttentionSystem.java Recurso de atención
- | | — DoorSystem.java Puertas con calor y bloqueo
- | | — DoorState.java OPEN, CLOSED, OVERHEATED
- | — útil/
- | | — MapLoader.java Parser del formato .txt
- | — view/
- | | — Theme.java Colores, fuentes, helpers
- | | — MainMenuFrame.java Menú inicial
- | | — MainFrame.java Ventana del juego
- | | — MapPanel.java Render del grafo y animatrónicos
- | | — HUDPanel.java Reloj, atención, calor
- | | — OfficePanel.java Botones de acción
- | | — DoorWidget.java Widget por puerta

— GameOverOverlay.java Pantalla de fin de partida

3. El mapa

Formato .txt

text

- SECTOR <id> <x> <y> "<nombre>"
- NODE <id> <sector_id> <local_x> <local_y> <tipo>
- LINK <a> # bidireccional
- LINK <a> > # direccional

OFFICE left|right <nodo>

Tipos de nodo: NORMAL, OFFICE, ENTRY_LEFT, ENTRY_RIGHT.

Modelo de grafo

- MapGraph mantiene el registro de nodos, sectores y cache de paths.
- BFS cacheado en shortestPath(from, to) y shortestPath(from, to, avoid). El cache se llena la primera vez y se reutiliza durante toda la partida.
- Validación al cargar: nodos huérfanos, entradas de oficina coherentes, un solo nodo OFFICE, conexión OFFICE↔ENTRY.

- Cada nodo puede tener múltiples vecinos. Los IDs de HOME y RESPAWN ya no son tipos de nodo: se definen por animatrónico en Main.

4. Sistemas de juego

4.1 Puertas y calor (DoorSystem)

Recurso compartido que sube con puertas cerradas y baja con puertas abiertas.

Estado	Velocidad
Ambas abiertas	-1.75/s
Una cerrada	+1.00/s
Ambas cerradas	+2.25/s
Enfriando (sobrecalentado)	-0.75/s

Al llegar a MAX_HEAT = 40.0 entra en OVERHEATED: ambas puertas se fuerzan a abrir y el jugador no puede operarlas hasta que el calor llegue a 0.

También expone:

- toggleLeft/Right — ignora si la puerta está bloqueada o el sistema sobrecalentado.
- raiseHeat(amount) — para eventos puntuales (shock de Endo sube +5).
- setLeftBlocked/RightBlocked — para que Chica y Bonnie reserven una puerta.

4.2 Atención (AttentionSystem)

Recurso alternativo a la batería clásica. Modelo de "tasa por frame":

- Luces: 0 luces $\rightarrow -1.00/s$, 1 luz $\rightarrow +0.75/s$, 2 luces $\rightarrow +1.50/s$.
- Aportes continuos (contribute(rate)) de animatrónicos: se acumulan y sobrescriben el decay base.
- Subidas puntuales (raise(amount)) de eventos: pizza (+3), shock (+5), Foxy bloqueado (+5), Freddy en fase de ruido (+0.25/s).

Si el total del frame es 0, aplica decay $-1.00/s$. Satura en 100.

4.3 Cámaras (GameState)

- Cambio de sector con cooldown de 1500 ms.
- Input buffer de 250 ms: si el jugador clickea durante el cooldown y faltan ≤ 250 ms, la intención se guarda y se ejecuta automáticamente al terminar.
- Cámaras bloqueables (setCameraBlocked): Bonnie en H16 bloquea hallway1 y hallway2. Si el jugador las está mirando, se lo expulsa a null.
- Apagar cámaras (turnOffCameras): deja activeSectorId = null. Necesario contra Golden Freddy.

4.4 Última vez visto

Cada animatrónico guarda lastSeenNodeId. GameState.updateLastSeen() corre cada tick:

- Si el animatrónico es visible para el jugador $\rightarrow$ marca su nodo actual.
- Si no es visible, pero la marca apunta a una zona visible $\rightarrow$ borra la marca (re-escaneaste y no está).
- Si no es visible y la marca apunta a zona no visible $\rightarrow$ la deja (sigue siendo útil).

"Visible" se decide en isNodeVisibleToPlayer:

- OFFICE $\rightarrow$ siempre.
- ENTRY_LEFT/RIGHT $\rightarrow$ si su luz está encendida.
- Resto $\rightarrow$ si su sector es el activo.
- Además, cualquier animatrónico con isForcedVisible() == true se considera visible.

5. Los 6 animatrónicos

Todos heredan de Animatronic y sobrescriben tick(dt, GameState).

5.1 Chica

- Home / Respawn: S1.
- Ruta: S1 → comedor → cocina. Deambula en cocina 15-25 s subiendo atención (+1/s). Luego P1 → P2 → O2 → O3 → H23 → H21 → PI.
- Pizza: si el jugador la ve en la cocina, puede ordenar pizza por +3 de atención. Chica cocina 13-22 s sin moverse ni hacer ruido, luego sale 20% más rápido.
- Puerta: ventana de 4 s si la puerta está abierta, 3 s si está cerrada. Si pasa de cerrada a abierta, grace de 1 s. Al bloquear, espera 3-6.5 s y mata.
- Observación: +15% más lenta mientras la mirás.
- Velocidad: 6-9 s entre movimientos.

5.2 Bonnie

- Home / Respawn: S3.
- Ruta: S3 → BP8 → party room (evitando BP5) → H11 → pasillo 1 → PD.
- Salto de nodo: si el siguiente está bloqueado, salta al siguiente del siguiente con coste de 1.5× intervalo.
- Desvío: 15% de probabilidad de ir a H16 antes de la puerta. En H16 bloquea las cámaras de los pasillos durante 10 s.
- Puerta: espera 4-7 s. Si la puerta está abierta al terminar, bloquea (puerta derecha), espera 2 s, mata. Si está cerrada, respawnea.
- Observación: +25% más lento.
- Velocidad: 4.5-7 s entre movimientos.

5.3 Freddy

- Home / Respawn: S2.
- Ruta: sortea entre PI y PD al inicio. Ruta más corta.
- Congelado: no avanza mientras su sector es el visible.
- Puerta:
 - Primeros 8 s: "fase de ruido", sube atención +0.25/s. Su badge parpadea cada 0.5 s si no tenés la luz prendida.
 - Luego 1-5 s "fase silenciosa": su badge queda fijo visible.
 - En cualquier momento, 3 s acumulados con la puerta abierta → mata.

- Cambio de ruta: si choca contra un obstáculo en los primeros 3 movimientos, cambia de puerta objetivo. Si ya está muy avanzado, retrocede.
- Velocidad: 8-11 s entre movimientos.

5.4 Foxy

- Home: E1.
- Fase entrada: timer que sube a +1/s si no lo mirás, baja a -0.7/s si lo mirás. Al llenar 9.3, avanza un nodo por $E1 \rightarrow E2 \rightarrow E3 \rightarrow E4 \rightarrow E5$.
- Rush: al llegar a E5 con timer lleno, va disparado a PD. 0.5 s por nodo, 0.8 s si tiene que saltar uno ocupado. Si no puede saltar, espera.
- Puerta: si llega con la puerta abierta $\rightarrow$ mata. Cerrada $\rightarrow$ sube +5 atención y respawnea 1 nodo después del respawn anterior (primera vez E2, siguientes E3).

5.5 Golden Freddy

- Home: BS4.
- Teletransporte: cada 14-21 s se teletransporta a un nodo aleatorio (no entrada, no sala de empleados, no ENTRY, no OFFICE).
- Chance acumulativa: cada teletransporte aleatorio suma +3% a la probabilidad de ir a la oficina. Se resetea al entrar.
- Mirarlo: si lo ves 6 s seguidos $\rightarrow$ mata.
- En la oficina: 5 s de ventana. Si acumulás 1.5 s con cámaras apagadas $\rightarrow$ se desvanece y entra en cooldown de 20 s.
- Si no lo apagás: mata a los 5 s.

5.6 Endo

- Home: EL2.
- Wandering: $EL2 \rightarrow EL1$ o $EL4$ cada 26-38 s. De $EL1$ va a $EL3$.
- Ventana de shock: al llegar a $EL3$ o $EL4$, 10-14 s para shockearlo desde la cámara de la sala de empleados. Si lo shockeás: +5 calor, +5 atención, vuelve a $EL2$.
- Si no lo shockeás:
 - Desde $EL3 \rightarrow$ HURRIED: cruza el pasillo saltando 2 nodos cada 4-6 s. En PI, 2 s $\rightarrow$ mata. Puerta cerrada $\rightarrow$ vuelve a O4 apagado.
 - Desde $EL4 \rightarrow$ SILENT: teleporta a PI sin hacerse visible. 5 s $\rightarrow$ mata. Puerta cerrada $\rightarrow$ vuelve a $EL2$.
- OFF en O4: cada 11 s, 35% de reactivarse en modo HURRIED.

- Luz + puerta abierta: si su puerta está abierta y prendés la luz izquierda → mata en 0.1 s. Hay que cerrar antes de mirar.

6. Interfaz

Layout del MainFrame

text



MapPanel

- Rejilla adaptativa: calcula bounding box de todos los nodos y escala.
- Nodos cuadrados con sombra, borde y highlight.
- Sectores con fondo redondeado, borde pulsante si es el activo.
- Links con color según si tocan el sector activo.
- Barra de puerta: línea gruesa verde/roja entre cada ENTRY y OFFICE.
- Estática durante 320 ms al cambiar de cámara (con scanline descendente).
- Cooldown ring: círculo pastel en la esquina superior derecha.
- Hover: borde azulado al pasar el mouse sobre un sector.
- Last seen: badge translúcido con borde punteado en el último nodo visto.

HUDPanel

- Reloj en formato H:MM calculado desde elapsedSeconds.
- Barra de atención con color según rango (cyan / violeta / magenta).
- Barra de calor con color según rango (verde → amarillo → naranja → rojo).

OfficePanel

- Widget de puerta izquierda + 3 slots de acción + widget de puerta derecha.
- Pizza (W): visible solo en cocina, habilitado solo si Chica puede aceptar.
- Shock (S): visible solo en sala de empleados, habilitado solo si Endo puede ser shockeado.
- Cámaras (C): visible siempre que haya cámara activa.

DoorWidget

- Estado de puerta: ABIERTA / CERRADA / SOBRECALENTADA / BLOQUEADA (morado).
- Estado de luz: APAGADA / ENCENDIDA.
- Botones PUERTA y LUZ con atajos y hover.

GameOverOverlay

- Glass pane con overlay negro al 68%.
- Título grande: GAME OVER o 6 AM.
- Subtítulo con razón de muerte.
- Hint: "Presioná R para reiniciar la noche · ESC para volver al menú".

7. Atajos de teclado

Tecla	Acción
Q	Toggle puerta izquierda
E	Toggle puerta derecha
A	Toggle luz izquierda
D	Toggle luz derecha

W	Ordenar pizza (en cocina)
S	Shockear a Endo (en sala de empleados)
C	Apagar cámaras
R	Reiniciar noche (solo en game over)
ESC	Volver al menú (solo en game over)

8. Decisiones técnicas clave

Concurrencia

- Un solo GameLoop con ScheduledExecutorService a 10 Hz (100 ms/tick).
- Tick único en GameState: ticea puertas, animatrónicos, atención y última vez visto dentro del mismo synchronized (lock). Esto elimina condiciones de carrera sin locks distribuidos.
- activeSectorId es volatile: escrito por el EDT (clicks), leído por el hilo del loop.
- La vista nunca muta el estado: usa AnimatronicView (record immutable) y getters que devuelven copias.

BFS cacheado

MapGraph.shortestPath usa un HashMap<String, List<String>> con key from>to (o from>to!avoid1,avoid2). El grafo es estático, así que el cache se llena en las primeras iteraciones y se reutiliza toda la partida.

Última vez visto

No se calcula en la vista. El GameState.updateLastSeen() centraliza la lógica y la vista solo lee. Esto evita tener dos definiciones distintas de "visible".

Input buffering

Cuando el jugador clickea durante el cooldown de cámara, si faltan ≤ 250 ms se guarda la intención en `bufferedSectorId`. Se consume en el próximo tick cuando el cooldown expira. Cambia la sensación de respuesta sin tocar la lógica del cooldown.

Bloqueo de puertas

Chica y Bonnie exponen `getBlockedDoorNodeId()`. `GameState.tick` recalcula cada frame qué puertas están bloqueadas. `DoorSystem.toggle` ignora clicks si la puerta está bloqueada. La vista solo pinta el estado.

Tema centralizado

Todos los colores y fuentes viven en `Theme.java`. Cambiar un color se reduce a editar una constante.

9. Estado actual y pendientes

Completo

- Modelo de grafo con parser de mapa.
- Sistema de puertas con calor.
- Sistema de atención.
- Cámaras con cooldown, input buffer, bloqueo y apagón.
- Los 6 animatrónicos con IA única.
- UI completa: mapa, HUD, oficina, menú, game over.
- Efectos visuales: estática, pulso de sectores, hover, blink.
- Last seen.
- Reinicio de noche y vuelta al menú.

Pendiente

- Al level por hora: los animatrónicos son igual de agresivos a las 12 AM que a las 5 AM. Falta escalar `moveInterval` y/o `aiLevel` con la hora.
- Panel de log visible: el log existe en `GameState.eventLog` pero no se muestra en la UI.
- Barra de progreso de hora: bajo el reloj.
- Sonidos: pasos, golpes, motor, jumpscare, 6 AM.

- Jumpscare animado: pantalla completa con zoom.
- Balance fino: jugar partidas completas y ajustar números.
- Modo debug: semilla visible, mostrar todos los animatrónicos siempre, acelerar el tiempo.

Bugs conocidos

- Ninguno crítico. Los warnings del IDE sobre campos "no leídos" (OfficePanel.state, active en MapPanel) son cosméticos.

10. Cómo correrlo

bash

- mvn clean compile

mvn exec:java

En VSC, usar la extensión Maven for Java y ejecutar la meta exec:java. No usar el botón de Run del editor porque no copia recursos al classpath.

En NetBeans, importar el proyecto como Maven existente y correr.

Ajustes de dificultad rápida

- Partidas más cortas: en GameState.java bajar SECONDS_PER_HOUR a 15.0.
- Todo más rápido/slow: pasar timeScale al constructor de GameLoop (new GameLoop(state, 2.0)).
- Chica test rápido: en Main.java, darle homeNodeId = "H21" para que arranque cerca de la puerta izquierda.
- Ver el mapa completo al perder: ya está implementado en paintSectorBackground (reveal = state.isGameFinished()).
-